SegQueue

Setup & acceptance testing guide

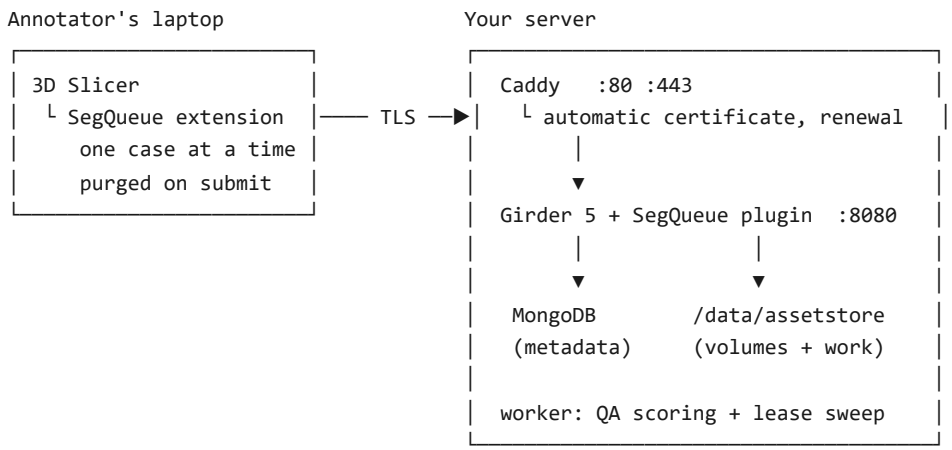
A self-hosted platform for producing per-segment coronary labels with a team of annotators: a Girder 5 server that owns the case pool, and a 3D Slicer extension that hands one annotator one case at a time and leaves nothing on their disk.

Asclepius · coronary segmentation

Covers server deployment, TotalSegmentator ingest, the Slicer client, and a full acceptance test. Server-only runbook: `docs/SERVER-SETUP.md`.

1 What you are setting up

Four processes on one Linux box, plus an extension on each annotator's machine. Nothing runs in a cloud account and nothing phones home.



CONTAINER	WHAT IT DOES	FAILS HOW
`caddy`	TLS termination and reverse proxy. Obtains and renews the certificate itself.	Nobody can connect; annotators see a TLS error.
`girder`	The API: accounts, tokens, resumable uploads, storage, plus the SegQueue plugin's queue, leases and review.	Everything stops, but no data is lost.
`mongo`	All metadata: cases, assignments, submissions, reviews.	Girder refuses to start. Restore from the nightly dump.
`worker`	Scores gold and duplicate submissions every 30 s; reclaims lapsed leases every hour.	Silent. Scores stop appearing and stranded cases stay stranded — check its log weekly.

1.1 The task

Four structures, fixed by the server and obeyed by the extension. The label values are the ones `configs/labels/coronary.yaml` uses, so a finished submission converts straight into a training set.

SEGMENT	LABEL	REQUIRED	WHERE IT RUNS
`left_main`	1	yes	Ostium at the left coronary sinus to the LAD/LCx bifurcation. Often under 10 mm.
`left_anterior_descending`	2	yes	Anterior interventricular groove, toward the apex. Diagonals included.
`left_circumflex`	3	yes	Left atrioventricular groove. Obtuse marginals included.
`right_coronary_artery`	4	no	Right atrioventricular groove. May be small or absent in a left-dominant system.

1.2 The three ideas worth knowing before you start

The server owns the labelling protocol. Segment names, label values, colours and the instruction text are server settings. The extension creates exactly those segments before the annotator can touch anything, so there is no

free-text segment name to get wrong across thirty people. Adding a structure mid-project is a settings change; nobody reinstalls anything.

Local data is a lease, not a library. The extension keeps one case in one managed directory and deletes it the moment the server confirms a submission — or on release, or at logout. The disk cost of the whole project on a student's laptop is one CT.

State lives on the assignment, not the case. A case can legitimately be in two annotators' hands at once (a blind duplicate), so "the case is assigned" is not a thing that can be said. Each *assignment* moves through `assigned → downloaded → submitted → approved`, with `rejected → assigned` for rework.

2 Before you start

YOU NEED	WHY, AND HOW MUCH
A Linux host with Docker and the Compose plugin	Ubuntu 22.04+ or equivalent. Compose v2 (`docker compose` , not `docker-compose`).
16–32 GB RAM	MongoDB's working set plus the page cache doing the real work on a file-heavy load. 16 is enough; 32 is comfortable.
A data disk, on RAID1 or a ZFS mirror	Budget **~300 MB per case**: about 1.5 TB for 5,000 CTs plus their segmentations. A single-disk failure part-way through is a year of undergraduate labour, not an inconvenience.
A DNS name pointing at the box, **or** a VPN	Off-campus annotators are the norm. See §3.6 — this is the decision most likely to bite you later.
The case data	By default the TotalSegmentator release on Zenodo — see §5. Bring-your-own CCTA works too, as NRRD or NIfTI: DICOM directories are not ingested, because converting once centrally beats thirty students meeting Slicer's DICOM import dialogue.
3D Slicer 5.8 on each annotator machine	No pip installs needed. The extension uses `requests` , which Slicer already bundles.
A UPS	MongoDB survives power loss far better with one. Cheap insurance on a machine holding the only copy of a term's work.

> **Just want to see it work first?** Skip to **§4 — Trying it on a laptop**. That path takes about five minutes, needs no domain name, no certificate and no data disk, and runs the same image the production stack does.

3 Production install

3.1 Get the code and configure

```
git clone https://github.com/christianGRogers/Asclepius.git /srv/segqueue
cd /srv/segqueue
git checkout segqueue-platform

cd deploy
cp .env.example .env
$EDITOR .env
```

SETTING	WHAT TO PUT
`SEGQUEUE_DOMAIN`	The public name, e.g. `segqueue.example.edu`. A hostname gets a real Let's Encrypt certificate. An `https://10.0.0.5` here gets a self-signed one instead — read §3.6 before choosing that.
`SEGQUEUE_TLS_EMAIL`	Where Let's Encrypt sends expiry warnings. Leave blank on the self-signed path.
`DATA_ROOT`	The mirrored data disk, e.g. `/srv/segqueue-data`. Mongo's files and the Girder assetstore both live under it. **Not the system disk.**
`CASE_SOURCE_ROOT`	Optional. A directory of CTs waiting to be ingested; mounted read-only at `/incoming` inside the container.
`MONGO_CACHE_GB`	Roughly a quarter of system RAM.

3.2 Start it

```
docker compose up -d --build
docker compose logs -f girder
```

You are looking for these three lines, in this order:

```
INFO:girder.models:Connecting to MongoDB: mongodb://mongo:27017/girder
INFO:girder.plugin:Loaded plugin "segqueue"
INFO:girder.asgi:Girder server running
```

> **If the plugin line is missing**, the API will answer but every `/segqueue/…` route will 404. See §9, first row.

3.3 Create the first account

Open `https://<your domain>/` and register. **The first user Girder sees becomes a site administrator**, so do this before telling anyone the address.

> On a brand-new database the plugin loads before any account exists, so it cannot yet create the `segqueue-annotators` and `segqueue-reviewers` groups — Girder requires an owner for a group. This is handled: the groups are created on the next restart, or the first time you create an annotator, whichever comes first. Nothing for you to do.

3.4 Create the assetstore

> **Nothing works until you do this, and the failure is confusing if you skip it.** Girder has no storage configured out of the box.

Admin console → **Assetstores** → **Create new filesystem assetstore**:

FIELD	VALUE
Name	`store` (anything)
Root directory	`/data/assetstore` — the path *inside the container*. Compose maps it to `$DATA_ROOT/assetstore` on the host.

The ingest CLI checks for this and refuses to start without it, rather than failing halfway through a 1.5 TB import.

3.5 Verify the plugin

```
curl -s https://<domain>/api/v1/system/version
curl -s https://<domain>/api/v1/describe | grep -o segqueue | head -1
```

Or open `https://<domain>/api/v1` in a browser: the generated API docs should show a **segqueue** section containing `project`, `next`, `mine`, `review/queue`, `stats` and `sweep`.

3.6 The certificate decision

Three options, in descending order of how much trouble they save you:

1. **A real DNS name.** Caddy gets a Let's Encrypt certificate and renews it forever. Nothing to install on annotator machines. Requires ports 80 and 443 reachable from the internet at certificate-issue time.
2. **A VPN with a real name.** Tailscale or WireGuard, with the box having a proper hostname inside it. Same as above, plus the server is never exposed. This is the recommended setup for a campus box.
3. **Self-signed.** Caddy issues its own. The Slicer extension will then **refuse to connect** until Caddy's local CA root is installed on each annotator machine — the root is at `/data/caddy/pki/authorities/local/root.crt` inside the caddy container. Workable for a handful of lab machines, painful for thirty students.

4 Trying it on a laptop

Docker Desktop on Windows or macOS, no domain, no certificate. This is the fastest way to see the whole system work, and it is exactly how the acceptance test in §7 was validated.

> Compose is not used here on purpose: MongoDB's storage engine is unreliable on a Windows bind mount. Running the containers directly with a Docker-managed volume sidesteps it entirely.

```
# from the repository root
docker network create segq-test
docker run -d --name segq-mongo --network segq-test mongo:7

docker build -f deploy/Dockerfile -t segqueue:test .

docker run -d --name segq-girder --network segq-test -p 8099:8080 \
  -e GIRDER_MONGO_URI=mongodb://segq-mongo:27017/girder \
  -e GIRDER_SERVER_MODE=production \
  segqueue:test

docker logs segq-girder | tail -5
```

Then `http://localhost:8099/` — register the first user, and create the assetstore at `/data/assetstore` exactly as in §3.4.

Make yourself some cases. They do not have to be real CTs: the server hashes and stores them without opening them, so random bytes exercise the same path.

```
docker exec segq-girder sh -c \
  'mkdir -p /tmp/incoming; for i in 1 2 3; do
     head -c 200000 /dev/urandom > /tmp/incoming/case00$i.nrrd; done'

docker exec segq-girder segqueue-ingest \n    --root /tmp/incoming --layout flat --target coronary
```

> **Git Bash on Windows** rewrites `/tmp/incoming` into `C:/Users/…` before Docker sees it, and the command fails with *"is not a directory"*. Prefix with `MSYS_NO_PATHCONV=1` , or use PowerShell.

Tear down with `docker rm -f segq-girder segq-mongo` and `docker network rm segq-test` . The Mongo data goes with it.

5 The data, and loading it

The default source is the TotalSegmentator release on Zenodo:

```
https://zenodo.org/records/10047292
```

About 1,200 whole-body CT studies, each **already segmented** into 117 structures. That presegmentation is the whole reason this dataset is a good starting point, because it lets the ingest decide two things from filenames alone without opening a single image.

```
<root>/s0011/ct.nii.gz
<root>/s0011/segmentations/heart.nii.gz              ← this case has a heart
<root>/s0011/segmentations/coronary_arteries.nii.gz  ← ... and a head start
<root>/s0011/segmentations/liver.nii.gz
<root>/s0012/ct.nii.gz
<root>/s0012/segmentations/femur_left.nii.gz         ← skipped: no heart
```

5.1 Only cases with a heart

Most of a whole-body dataset is legs, heads and abdomens with no coronary anatomy in the field of view at all. Assigning those spends the one resource this project is short of — annotator hours — on scans that cannot be annotated. A case qualifies if it carries any of `heart`, `heart_myocardium`, `heart_atrium_left`, `heart_atrium_right`, `heart_ventricle_left` or `heart_ventricle_right`: v2 ships the first, v1 the rest, and both releases are in circulation.

Qualifying cases also send their heart mask to the client, which uses it to centre the slice views on arrival.

5.2 A coronary mask, when the case has one

If a case carries `coronary_arteries`, it travels with the case and the extension loads it as a helper segment. It is a **binary lumen**, not per-branch labels — so it is a head start rather than an answer. The annotator splits an existing tree into four branches instead of drawing one, which is minutes of work instead of an hour.

> The base Zenodo release does **not** include coronary masks; TotalSegmentator's `coronary_arteries` task is licensed separately. If your copy has them — from that task, or from your own earlier model — ingest picks them up with no extra flags. If not, everything still works and annotators draw from scratch.

> Helper masks cannot be submitted. The export copies only the project's own four segments into a throwaway segmentation, and then refuses outright if any label value outside the protocol appears in the result. Shipping the dataset's own mask back as a student's work would corrupt the training set while every dashboard number looked healthy, so it is prevented structurally rather than by convention.

5.3 Running the ingest

```
# always first: changes nothing, tells you what the filter did
docker compose exec girder segqueue-ingest --root /incoming --dry-run

# then
docker compose exec girder segqueue-ingest --root /incoming --target coronary
```

A real run against a five-case test tree — yours will be larger:

```
Scanned /incoming
  5 case(s) with a CT
  3 with a heart segmentation, 2 without
  1 with a pre-existing coronary mask
  0 with an expert reference

  + s0001  [coronary seed, heart]
  + s0002  [heart]
  + s0003  [heart]

ingested 3 case(s): 3 with a heart mask, 1 with a coronary head start, 0 gold.
```

The *rejected* count is printed on purpose. A scan that quietly returns 12 cases out of 1,200 looks identical to a correct one until somebody asks why the project finished early.

FLAG	MEANING
`--root`	Directory holding one subdirectory per case.
`--layout`	`totalsegmentator` (default), or `flat` for a plain directory of volume files.
`--all-cases`	Ingest cases with no heart segmentation too. Off by default.
`--no-seed`	Do not ship pre-existing coronary masks. Use this to measure unaided annotation time.
`--no-region`	Do not ship heart masks. The extension then cannot centre the view.
`--gold-root`	A **separate** directory of expert per-branch references — see below.
`--target`	Recorded on each case. Defaults to `coronary`.
`--priority`	Higher is served first. Useful for a pilot batch.
`--limit`	Stop after N new cases. Good for a first run.
`--dry-run`	Report and change nothing.

Ingest is idempotent by case name. An interrupted import resumes by running the same command again — which matters, because that is exactly the kind of job that gets interrupted. The SHA-256 of every CT is computed here, once, and every later transfer verified against it, so a file corrupt on arrival is caught by the first annotator rather than blamed on their connection.

5.4 Gold-standard cases

```
docker compose exec girder segqueue-ingest \
    --root /incoming --gold-root /incoming-gold --target coronary
```

Expert references live in a parallel directory, never beside the volumes. A mis-scoped `--root` that swept them into the case pool would hand annotators the answers to the very cases used to measure them, and the failure would be invisible in the metrics. Aim for about **5%** of the pool.

6 People and the Slicer client

6.1 Create annotators

```
curl -u admin1 -X POST "https://<domain>/api/v1/segqueue/users" \
     -d login=student01 -d email=student01@example.edu \
     -d firstName=Ada -d lastName=Lovelace \
     -d password=... -d quota=200
```

Or through Girder's own admin UI: create the user, then add them to the `segqueue-annotators` group. Reviewers additionally join `segqueue-reviewers`; site admins review by right.

OPERATION	CALL
Change a quota	`PATCH /segqueue/users/<id>?quota=300` (`-1` removes the cap)
Grant reviewer	`PATCH /segqueue/users/<id>?reviewer=true`
Student left the course	`PATCH /segqueue/users/<id>?disabled=true` — also releases every case they were holding

6.2 Install the extension

Build the package once per release, on any machine with Python. No CMake and no Slicer needed to build it:

```
python slicer/build-extension.py
# -> dist/SegQueue-0.1.0-Slicer-5.8.zip
```

Then each annotator: **Extensions Manager → Install from file**, pick the zip, restart, and open **Segmentation → SegQueue**. No pip install, no build tools and no admin rights — the module uses `requests`, which Slicer already ships, and the shared `segqueue` package is vendored into the archive.

> **One dependency: SegmentEditorExtraEffects.** It provides the *Draw tube* effect, which is the tool this module is built around. Install it from the Extensions Manager's **Install Extensions** tab on each annotator machine. The package declares the dependency, but a local *Install from file* does not resolve dependencies — so the panel checks at startup and names it if missing, and Paint still works meanwhile.

> **Rebuild after any change to `src/segqueue`.** The copy inside the archive is what annotators actually run. A stale one speaks an older wire protocol than the server, and the server will refuse it — politely, but it will refuse it.

Two other routes exist, both running the module straight out of a checkout so that `git pull` updates it. Useful while developing, awkward to give to thirty students:

- Slicer → **Edit** → **Application Settings** → **Modules**, drag `slicer/SegQueue` into **Additional module paths**, restart. Keep `slicer/` and `src/` side by side — the module imports `segqueue` from two directories above itself.
- In the Python Console (**View** → **Python Console**), one line that does the same and checks the layout first:

  ```
  exec(open(r"C:\Asclepius\slicer\install-segqueue.py").read())
  ```

6.3 What an annotator does

1. Enter the server address and log in. The address is remembered; the password never is.
2. **Get next case.** The volume downloads, its checksum is verified, the project's segments appear already named and coloured, the view lands on the heart at CTA window/level, and any coronary mask the case carries is loaded ready to split.
3. Pick a vessel with **1–4**. Draw the artery with **Q** as a series of tube sections, narrowing the radius as it tapers and pressing **A** after each; tidy up with **W**.
4. **Validate & submit.** Problems are listed in plain language before anything is uploaded. On success the local copy is deleted.

Drafts autosave every two minutes, and the elapsed-time clock is accumulated into a file rather than held in the panel, so a Slicer crash at minute forty comes back to the work with the clock intact.

6.4 The vessel tools

Slicer's Segment Editor can already do every one of these. It is worth wrapping because it cannot do them *for this task*: a first-year undergraduate should not have to work out which of twenty effects segments a 3 mm vessel, nor scroll a segment list to change branch two hundred times an hour.

CONTROL	WHAT IT DOES, AND WHY IT IS THERE
1 – 4	Select the branch. Each button carries the vessel's colour, its one-line hint, and a tick once it has voxels in it — so "have I done the RCA yet" is answered by looking, not by clicking through segments. Built from the server's segment list, so a fifth branch is a settings change.
Q Draw tube	**The main tool.** A coronary artery is a tube: click a handful of points down its centreline and the effect sweeps a tube along them. Faster than slice-by-slice painting, and far more consistent between annotators, because the cross-section is defined by one number instead of by whoever is holding the mouse.
A Apply tube	Adds the section you placed to the vessel and arms the next one. **Sections accumulate**, which is what lets one artery be drawn as several: a wide section proximally, narrower ones as it tapers. (The effect on its own replaces the segment each time, which would erase every section but the last.)
Tube radius	Live. Vessels taper — a left main is around 2 mm in radius and a distal LAD under 1 — so change it between sections, or while placing points, and the preview reshapes with it. Remembered between sessions. Below about 0.75 mm the tube is thinner than a voxel on 1.5 mm data and starts to render patchily; finish those distal stretches with Paint.
W Paint	Touch up what the tube missed: an ostium, a bifurcation, a short branch that does not deserve its own tube. Sized in millimetres rather than screen pixels, so it stays the same size as you zoom.
Only paint inside that mask	Confines every effect to the case's existing coronary mask, when it has one. A fast, sloppy stroke then still produces a clean vessel edge.
Only paint over opacified lumen	Restricts editing to 150–1000 HU, so the brush cannot spill into myocardium or fat.
Add the whole mask to this vessel	Unions the entire pre-existing tree into the selected branch, to be trimmed down. Useful for whichever branch dominates the tree.
Centre on heart / CTA window/level	Both are applied automatically when the case opens; the buttons put them back after the annotator has explored.
Show in 3D	Builds a surface of what has been drawn. The fastest way to spot a branch that stops early or a stray blob. Helper masks are kept out of the 3D view, since a solid heart would hide the tree.

Only these two effects get buttons. Erase, scissors, islands and the rest are still in the Segment Editor immediately below — they are simply not what a coronary needs often enough to occupy the top of the panel.

Two behaviours are set for the annotator and not exposed: branches never overwrite each other (so painting the LCx cannot silently erase half the LAD), and the helper masks are excluded from the submission by construction.

7 Acceptance test

Run this once after any install or upgrade. It drives the real HTTP API with the *same client the Slicer extension uses*, so a pass means the extension will work.

```
python tests/segqueue_e2e.py --url http://localhost:8099
```

It needs at least one unclaimed case in the pool (two to also check the concurrency guard), and it is idempotent — a second run reuses the accounts the first one made. Expected tail:

```
============================================================
34 passed, 0 failed
============================================================
```

7.1 What each check proves

CHECK	WHAT WOULD BE BROKEN IF IT FAILED
a filesystem assetstore exists	§3.4 was skipped; no upload can succeed.
the server sends the labelling protocol	The project setting is empty; annotators would draw free-text segments.
the server names an upload folder	Annotators have nowhere they are permitted to write.
the case flavour is hidden from the annotator	Gold and duplicate cases would be identifiable, making every QA number meaningless.
the volume matches its checksum	Silent data corruption in transit — the failure mode requirement N3 exists to prevent.
empty / stray / off-protocol / resampled submissions refused	Bad work would reach reviewers, and the training conversion would receive segments it cannot interpret.
a corrupted upload is refused	The server is trusting the client's checksum instead of recomputing it.
the first case goes to a human	The training gate is off; a new annotator could accumulate a hundred cases of the same misunderstanding.
a rejection with no comment is refused	Rework would go back to a student with no idea what to change.
reworking renews the lease and bumps the attempt	Rework would expire mid-fix, or overwrite the original attempt's record.
a claimed case is not handed to a second annotator	The atomic claim is broken — two people would silently duplicate each other's work.

7.2 The unit suites

```
pytest                                  # 366 tests, no server needed

# The concurrency tests need a real MongoDB: the whole point is that the
# atomic claim survives thirty annotators starting at once, which no fake
# can demonstrate.
docker run -d --rm -p 27099:27017 mongo:7
SEGQUEUE_TEST_MONGO=mongodb://localhost:27099 pytest tests/test_segqueue_server.py
```

Tests needing the TotalSegmentator dataset are marked `needs_data` and those needing a database `needs_mongo`; both skip cleanly, so the suite runs on a bare checkout.

7.3 Manual walkthrough in Slicer

The automated test covers everything below the UI. This covers the UI. Budget fifteen minutes.

#	DO THIS	EXPECT
1	Open **Segmentation → SegQueue**, enter the server, log in.	Status shows your login and remaining quota. The **Server** section collapses. Instructions appear.
2	Press **Get next case**.	A progress bar, then the CT in the slice views, with the project's segments listed — named, coloured, empty.
3	Check the **Case** panel.	Case name, "attempt 1", and a due date. The clock starts counting.
4	Press **Validate & submit** with nothing segmented.	Refused, listing each empty required segment by name. Nothing uploads.
5	Paint a few voxels in one segment only, submit again.	Still refused — the painted one now passes, the rest are still named as empty.
6	Paint all required segments properly. Press **Check without submitting**.	"All checks passed", in green. Optional segments left empty appear as amber warnings, not errors.
7	Quit Slicer without submitting. Reopen, log in.	Offered the case back. The segmentation is as you left it and **the clock carries on** from where it stopped.
8	**Validate & submit**, confirm.	Upload progress, then "Submitted." Check the cache directory (`~/.segqueue/cases`) — **it is empty**.
9	As a reviewer, expand **Review → Refresh queue**.	The submission is listed with annotator, attempt, auto score and flags.
10	Claim it, press **Reject** with an empty comment.	Refused before anything is sent.
11	Reject with a real comment.	Scene clears; the queue refreshes without it.
12	Back as the annotator, press **Get next case**.	You get **the same case back**, attempt 2, with the reviewer's comment in an orange box above the instructions — not a new case.
13	Fix and resubmit; approve as reviewer.	`GET /segqueue/stats` shows `cases.complete` incremented.

7.4 Things worth breaking deliberately

BREAK THIS	SHOULD HAPPEN
Pull the network cable mid-upload, reconnect.	The upload resumes from the server's real offset — not from zero, and not by resending a chunk that already landed.
Stop the server mid-download.	The partial file is deleted, not left looking complete. The message names the byte counts.
Ask for a second case while holding one.	Refused: finish or give back the one you have.
Set `lease_days` to a tiny value, wait, then `POST /segqueue/sweep?dryRun=true`.	The held case is listed as reclaimable, with a reason. Without `dryRun` it returns to the pool.
Fill the annotator's disk.	The download is refused *before it starts*, naming how much is needed and how much is free.
Disable an annotator holding two cases.	Both cases are released and immediately available to others.

8 Configuration

Two settings hold everything tunable. Both are validated on write — a bad value is rejected immediately rather than failing when the fiftieth annotator asks for a case.

```
# read
docker compose exec girder sh -c 'cat > /tmp/s.py <<EOF
from girder.models.setting import Setting
import json
print(json.dumps(Setting().get("segqueue.policy"), indent=2))
EOF
girder shell /tmp/s.py'
```

```
# write
docker compose exec girder sh -c 'cat > /tmp/s.py <<EOF
from girder.models.setting import Setting
policy = Setting().get("segqueue.policy")
policy["base_review_rate"] = 0.30
Setting().set("segqueue.policy", policy)
EOF
girder shell /tmp/s.py'
```

8.1 `segqueue.policy`

KEY	DEFAULT	WHAT IT CONTROLS
`training_gate_cases`	5	Every annotator's first N cases are reviewed by a human, whatever the sampling says.
`base_review_rate`	0.20	Fraction of ordinary work reviewed once past the gate.
`trusted_review_rate`	0.10	Rate for annotators with a long clean streak.
`trusted_after_clean`	20	Consecutive approvals needed to earn the trusted rate.
`probation_cases`	3	Cases reviewed at 100% immediately after any rejection.
`gold_rate`	0.05	Share of assignments that are gold-seeded.
`duplicate_rate`	0.05	Share that are blind duplicates of already-approved cases.
`gold_first_case`	true	Make an annotator's very first case a gold one, so you have a baseline for them from day one.
`lease_days`	7.0	How long a case may be held before the sweeper reclaims it.
`stale_heartbeat_hours`	72.0	Silence after which a case is reclaimable even inside its lease.
`max_concurrent`	1	Cases one annotator may hold at once. Raising it also raises the extension's local cache cap.
`gold_dice_flag`	0.70	Mean Dice against the expert reference below which a submission is pulled back for a human.
`duplicate_dice_flag`	0.70	Same, for agreement between a duplicate pair.

Gold and duplicate rates are drawn from **one** random number partitioned across the two, so they can never overlap and a case is never both.

8.2 `segqueue.project`

`name`, `instructions` (Markdown, shown above the segment list), and `segments`. Each segment has `name`, `label`, `color`, `required` and `hint`. Duplicate names *and* duplicate label values are both rejected on save.

The shipped default matches this repository's phase 1 task:

NAME	LABEL	REQUIRED
`left_main`	1	yes
`left_anterior_descending`	2	yes
`left_circumflex`	3	yes
`right_coronary_artery`	4	no — may be small or absent in a left-dominant system

> These label values must match `configs/labels/coronary.yaml`. If they drift, `segtrain convert` will happily build a training set with the wrong structures under the right names — the worst possible outcome, and silent.

9 Troubleshooting

SYMPTOM	CAUSE AND FIX
Every `/segqueue/…` route 404s	The plugin did not load. Check `docker compose logs girder | grep plugin`. Usually the image was built without the repository root — see the next row.
`ImportError: girder-segqueue needs the segqueue package`	The plugin is installed but the shared package is not. `girder-segqueue` deliberately does not declare it as a dependency, because the distribution providing it is named `segtrain` and an unrelated project owns that name on PyPI. Install the repository root first: `pip install /repo` before `pip install /repo/server`. The supplied Dockerfile already does.
`Girder has no assetstore configured` from ingest	§3.4 was skipped.
Uploads fail with a permission error	The annotator is not in `segqueue-annotators`. That group is the only place they are granted write access, and only to the drop-box folder.
Extension: *"Could not reach the server… If you are off campus, check that the VPN is connected."*	Exactly what it says — or a self-signed certificate whose CA root is not installed. See §3.6.
Extension: *"the server returned something that is not JSON"*	A captive portal or proxy is intercepting. Common on hotel and campus guest wifi.
Extension: *"Could not import the segqueue package"*	The module was loaded from a copy outside the repository. It imports `segqueue` from the sibling `src/` directory; both must travel together.
Annotator gets "no cases available" while cases clearly remain	Normal near the end: every remaining case has already been shown to them, and nobody is offered the same case twice. Check `GET /segqueue/stats`.
Submissions accepted but never scored	The worker container is down or lacks the scoring extras. `docker compose logs worker`.
Progress has stalled and cases are held by nobody	`POST /segqueue/sweep?dryRun=true` to see them. The worker sweeps hourly on its own; the endpoint is for when you do not want to wait.
Git Bash: *"--root C:/Users/… is not a directory"*	MSYS is rewriting container paths. Prefix the command with `MSYS_NO_PATHCONV=1`, or use PowerShell.
Mongo crashes on Windows	Its data directory is on a bind mount. Use a Docker-managed volume (§4) — this affects laptop testing only.

10 Running it

10.1 Watching the project

QUESTION	CALL
How is it going?	`GET /api/v1/segqueue/stats` — burn-down, 14-day velocity, projected finish date
Who is doing what, how fast?	`GET /api/v1/segqueue/stats/annotators` — throughput, median time per case, QA pass rate
Anything stranded?	`POST /api/v1/segqueue/sweep?dryRun=true`
Is QA keeping up?	`docker compose logs --tail=50 worker`

10.2 Backups

```
0 2 * * * RESTIC_REPOSITORY=/mnt/backup/segqueue \
          RESTIC_PASSWORD_FILE=/root/.restic-pass \
          /srv/segqueue/deploy/backup.sh >> /var/log/segqueue-backup.log 2>&1
```

The script dumps MongoDB *first*, into the tree restic then reads, so the metadata snapshot is never newer than the files it describes. The other order produces a backup that references submissions it does not contain — which looks fine until the day you restore it.

Restore is `mongorestore --archive --gzip` followed by a restic restore of the assetstore, in that order. **Test it once, on purpose, before you need it.** An untested backup is a hypothesis.

10.3 Upgrading

```
cd /srv/segqueue && git pull
cd deploy && docker compose up -d --build
python ../tests/segqueue_e2e.py --url https://<domain>
```

The extension–server protocol is versioned. If a release changes it incompatibly, old extensions receive a refusal telling the student to update, rather than writing subtly wrong data for a month before anyone notices. Annotators update by pulling the repository; there is nothing to reinstall.

11 Known gaps

Stated plainly, so you do not discover them at an awkward moment.

- **No export to the training layout.** Approved submissions sit in the assetstore as label volumes on the source grid — the right contents — but reshaping them into the `<case>/segmentations/` layout `segtrain convert` reads is manual today.
- **The Slicer panel has no automated test.** The network layer, the local cache, the state machine and the sampling policy are all unit-tested without Slicer; the Qt widgets are exercised only by §7.3.
- **Gold and duplicate scoring has not run on real label volumes.** The metric code is the same as the training pipeline's and is unit-tested, but the end-to-end path from a real `.seg.nrrd` to a Dice number has not been through a live case.
- **Nothing has been run at 5,000-case scale.** The 30-annotator claim race is tested against real MongoDB; sustained load over a term is not.
- **No admin web UI.** Admin operations are REST calls and Girder's stock console. Deliberate — a dashboard is a lot of code to maintain for one grad student, and `stats` answers the questions that matter.

> **If §7 passes and §7.3 behaves as described, the system is working.** Everything else in this document is either setup that precedes those tests or operations that follow them.